Some Practice Questions

File IO:

Q1. Assume that barfoo.txt contains “ABCDEF” 6 characters. What will the output from the code?

```
int main() {
        int fd1, fd2;
        char c;
        fd1 = open("barfoo.txt", O_RDONLY, 0);
        fd2 = open("barfoo.txt", O_RDONLY, 0);
        read(fd1, &c, 1);
        dup2(fd2, fd1);
        read(fd1, &c, 1);
        printf("c = %c", c);
        exit(0);
}
```

c = B

Q2. If a user runs a following command on a file that has 444 permissions, what will happen?

```
rm file1.txt
```

Q3. Considering File Control Block (FCB) supports 10 direct pointers, 3 single indirect pointers, 3 double indirect pointers and 1 triple indirect pointer. Calculate the maximum size of the file supported by this FCB when each of the data block address is 32 bits. Assume each data block to be of 2KB (2048 bytes).

Q4. When ls -i is executed, following output is generated, what can you say about the output?

1000 file1.txt
1000 file2.txt
1001 file3.txt → file1.txt

a. file1.txt and file2.txt have same inode numbers so they are soft links
b. file1.txt and file2.txt have same inode numbers so they are hard links
c. If we delete file1.txt, we can still get the content of the same file by executing cat file3.txt
d. If we delete file1.txt, we can still get the content of the same file by executing cat file2.txt
e. None of the above

Q5. What will be content of the Process File Descriptor (PFD) table and Systems Wide File Open (SWFO) table upon executing following code?

```
int main() {
        int fd1, fd2;
        char c;
        fd1 = open("barfoo.txt", O_RDONLY, 0);
        fd2 = open("barfoo.txt", O_RDONLY, 0);
        read(fd1, &c, 1);
        dup2(fd2, fd1);
        read(fd1, &c, 1);
        printf("c = %c", c);
```

```
        exit(0);
}
```

Assume that the initial entries of PFD and SWFO are shown below.

Index	SysFD Ptr
1	fd1=10
2	
3	
4	

SysFD	Offset	RefCnt	inode Ptr
10	0	1	1000

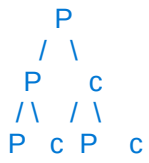
Process Management:

Q1. How many times 'ls' command will get executed in the below code?

```
int main()
{
fork();                         // line 1
fork();                         // line 2
execlp("ls", "ls", NULL);       // line 3
fork();                         // line 4
execlp("ls", "ls", NULL);       // line 5
}
```

Q2. Draw a binary to show the number of processes created by following code. What is the count of number of processes created?

```
int main(void)
{
        fork() && fork();
}
```



4 process

Q3. What value of i variable be printed by parent and the child process from the below code?

```
int i=0;
int main(void)
{       int pid, childpid, status;
        i++;
        pid = fork();
        i++;
        if (pid != 0)       // parent process
        {
```

```
                i++;
                childpid = wait(&status);
                i = i + WEXITSTATUS(status);
                printf("parent: i = %d\n", i);
        }
        else                    // child process
        {
                i += 2;
                printf("child: i = %d\n", i);
                exit(10);
        }
}
```

Q4. List out the process states in the same order **the child process** goes through when the following code gets executed.

```
int main()
{
        if (fork() == 0)
        {
                int fd; char buf[100000]
                fd = open("large.txt",O_RDONLY);
                read(fd,buf,len(buf));
                exit(10)
        }
        else
        {
                while(1)
                        sleep(5*60);
        }
}
```

Q5. Write a master-slave implementation for the following situation.

1. Main program accepts 5 filenames of text files as command line argument.
2. You need to create 5 child processes and pass each of the 5 filenames as a command line argument to each of the 5 child process. For example first child will get the filename 1, second child will get filename 2 and so on.

3. Child process will execute wc command for the filename provided and display the output from the wc command.

A2) The rm command is used to remove files, and requires write permissions on the directory containing the file to be removed, not on the file itself.

Since the file file1.txt has permissions set to 444, it means that it has read-only permissions for all users, including the owner, the group, and others. This implies that the file cannot be modified, including deleted by any user, as they do not have write permissions.

Therefore, if a user runs the command rm file1.txt on this file, the command will fail with a "Permission denied" error message, and the file will not be deleted.